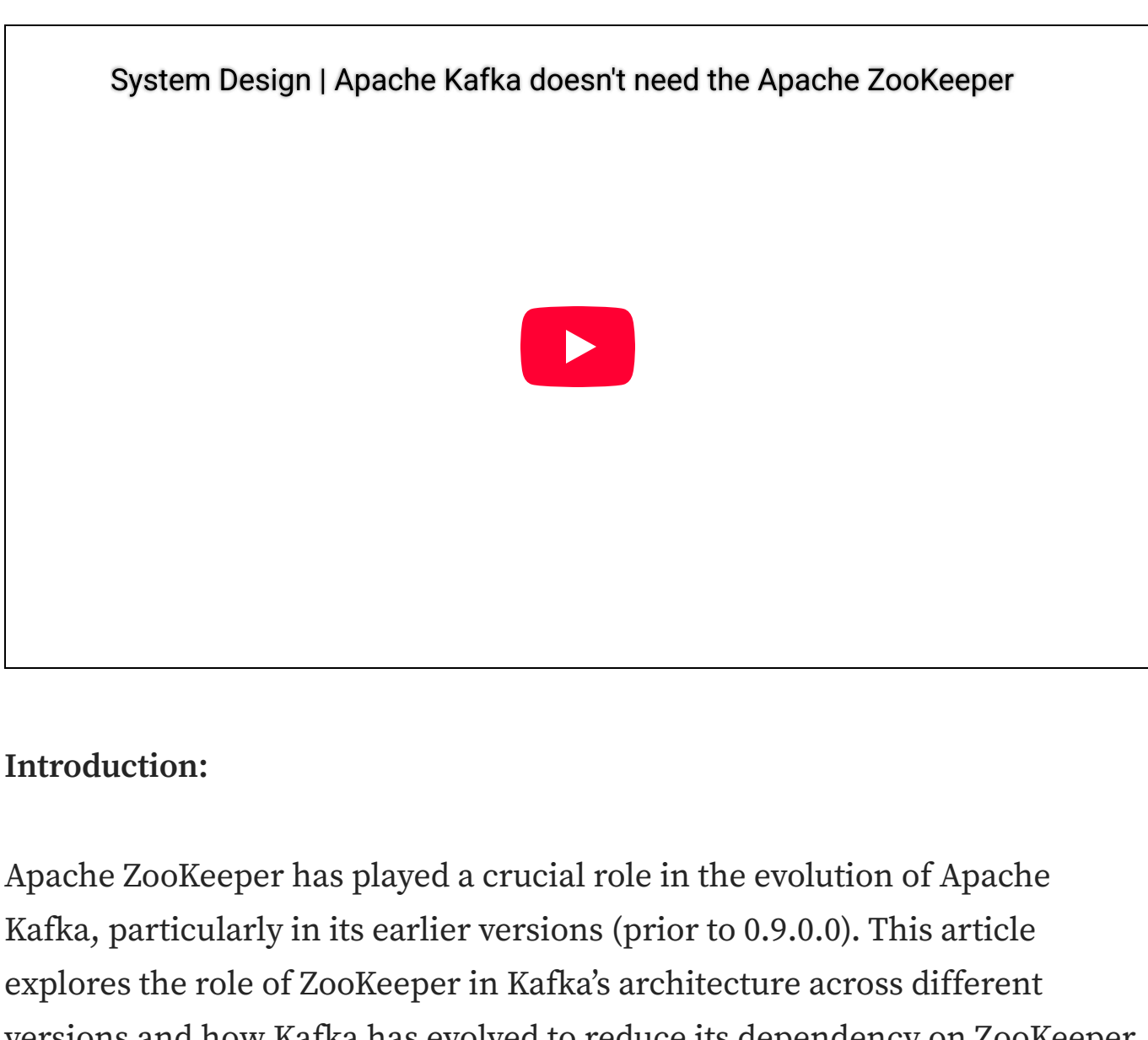


Understanding the Evolution of ZooKeeper in Apache Kafka

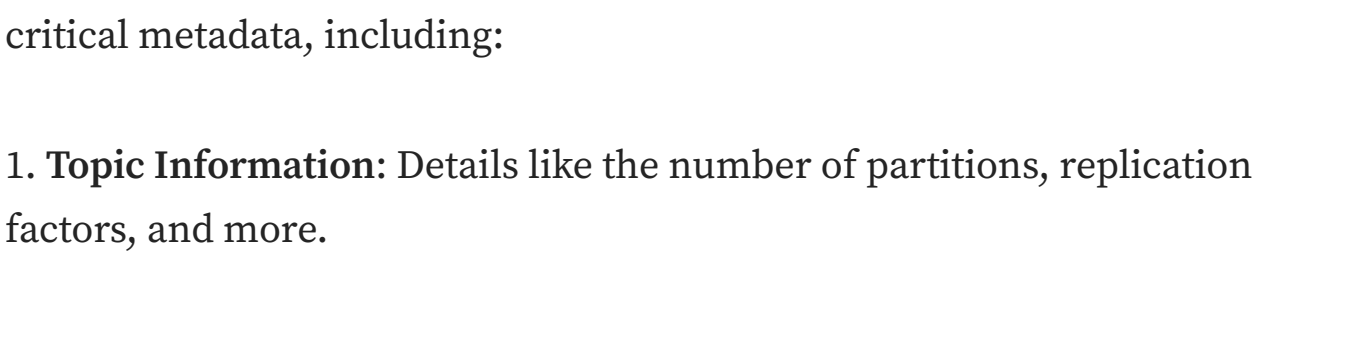
Roopa Kushtagi Follow 6 min read · Sep 26, 2023

Several topics like this are discussed on [my YouTube channel](#). Please, visit. Appreciate your support.



Introduction:

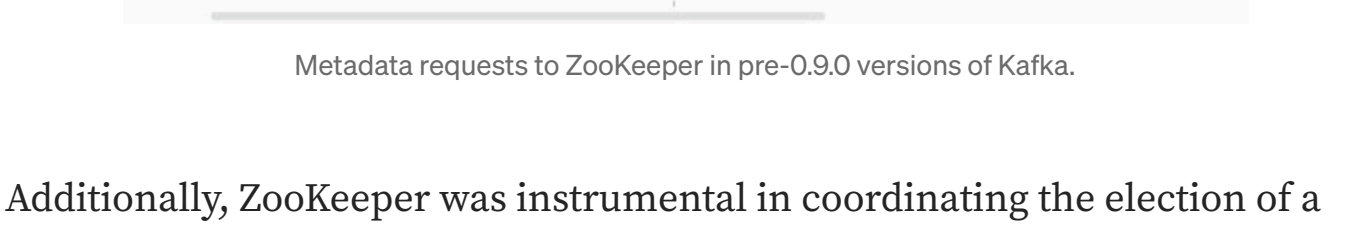
Apache ZooKeeper has played a crucial role in the evolution of Apache Kafka, particularly in its earlier versions (prior to 0.9.0.0). This article explores the role of ZooKeeper in Kafka's architecture across different versions and how Kafka has evolved to reduce its dependency on ZooKeeper.



ZooKeeper's Role in Pre-0.9.0.0 Kafka

Before version 0.9.0.0, Kafka relied on ZooKeeper for storing and managing critical metadata, including:

- Topic Information:** Details like the number of partitions, replication factors, and more.
- Broker Information:** Information about Kafka brokers, including IP addresses and port numbers.
- Consumer Group Information:** Information about which consumers are subscribed to specific topics.



Additionally, ZooKeeper was instrumental in coordinating the election of a controller broker, responsible for cluster management and partition rebalancing.

The Pre-0.9.0.0 Kafka Communication Process

In earlier Kafka versions, the **old Kafka protocol** was designed for communication between ZooKeeper and Kafka brokers. It was **not designed for direct communication** between producers and consumers and Kafka brokers. Producers and consumers communicated through ZooKeeper as follows:

- Producers connected to ZooKeeper to discover brokers in the cluster before sending messages.
- Once the producer had the latest cluster metadata, it could start sending messages to brokers. However, the producer still needed to go through ZooKeeper to coordinate the election of a controller broker and to ensure that all of the producers had the same view of the cluster state.
- Consumers also connected to ZooKeeper to identify brokers and gather information about topics they subscribed to before polling brokers for messages.

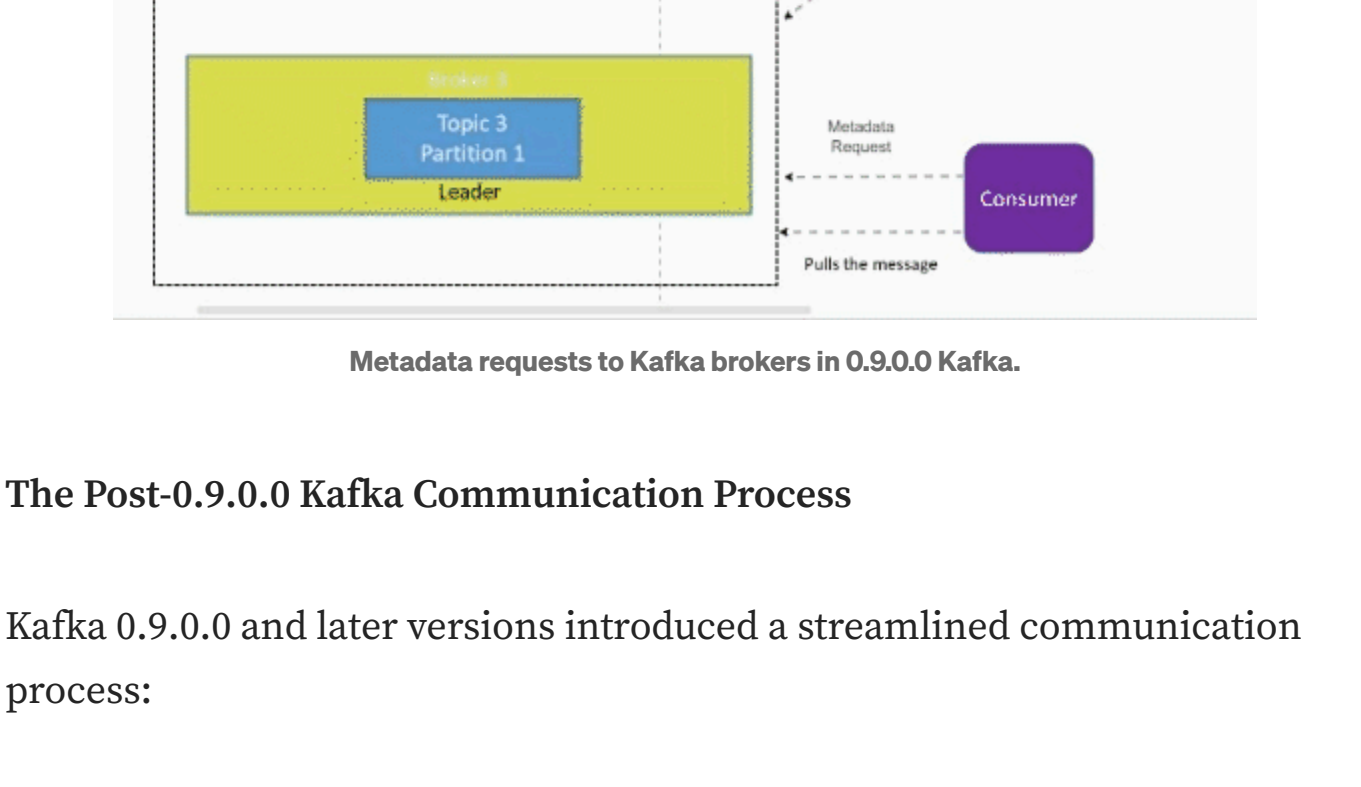
However, this architecture had drawbacks:

- Complexity:** Producers and consumers had to understand and interact with ZooKeeper multiple times.
- Performance:** Communication had to pass through ZooKeeper, which could impact performance.
- Scalability:** ZooKeeper presented a single point of failure.

The Shift in Kafka 0.9.0.0

With the release of Kafka 0.9.0.0, a significant shift occurred. While ZooKeeper continued to manage metadata, ZooKeeper was no longer required for producers and consumers to communicate with Kafka brokers directly.

This change was enabled by the introduction of a **new Kafka protocol**, a binary protocol for **direct communication** between brokers, producers, and consumers. In previous versions, producers and consumers needed to interact with Kafka brokers via ZooKeeper.



The Post-0.9.0.0 Kafka Communication Process

Kafka 0.9.0.0 and later versions introduced a streamlined communication process:

- Producers connected directly to brokers to send messages.
- Consumers also connected directly to brokers to subscribe to topics.
- Brokers then transmitted messages from subscribed topics to consumers.

Advantages of the New Architecture

The new Kafka protocol brought several benefits:

- Simplicity:** Producers and consumers no longer need to be aware of ZooKeeper.
- Performance:** Direct communication between brokers, producers, and consumers improved efficiency.
- Scalability:** The removal of ZooKeeper as a single point of failure enhanced scalability.

ZooKeeper's Ongoing Role in Kafka

Despite these changes, ZooKeeper remains a vital part of Kafka. It's recommended to run ZooKeeper alongside Kafka in production environments because it provides:

- Fault Tolerance:** ZooKeeper is highly available and can tolerate node failures.
- Consistency:** It offers a consistent view of the cluster to all clients.
- Coordination:** ZooKeeper facilitates coordination among distributed applications, including Kafka.

Examples of ZooKeeper Usage in Kafka 0.9.0.0:

- Registering new brokers in the cluster.
- Registering new topics created by producers.
- Watching for changes in topic metadata by consumers.
- Handling broker failures and triggering partition rebalancing.
- Supporting the controller broker's cluster state tracking.

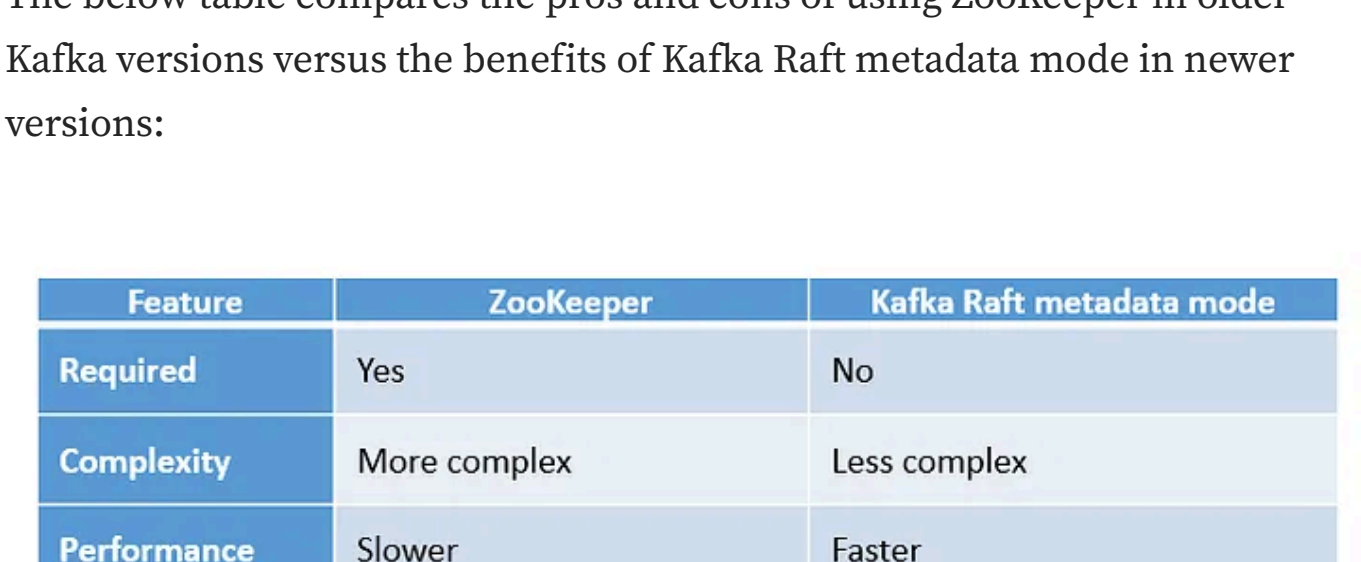
Kafka Raft Metadata Mode

Beginning with Kafka version 2.8.0, Apache Kafka introduced Kafka Raft metadata mode as an alternative to ZooKeeper. Kafka Raft metadata mode eliminates the need for a separate ZooKeeper cluster and offers advantages like reduced complexity, improved performance, and simplified operations.

Kafka's internal Raft quorum is a distributed consensus algorithm that is used to store and manage metadata. The Raft quorum consists of a set of brokers, and each broker maintains a copy of the metadata.

The Raft quorum works by electing a leader broker. The leader broker is responsible for replicating the metadata to all of the other brokers. When the leader broker receives a request to change the metadata, it appends the change to its log and then replicates the log to the other brokers. Once a majority of the brokers have acknowledged the change, the leader broker commits the change to the metadata.

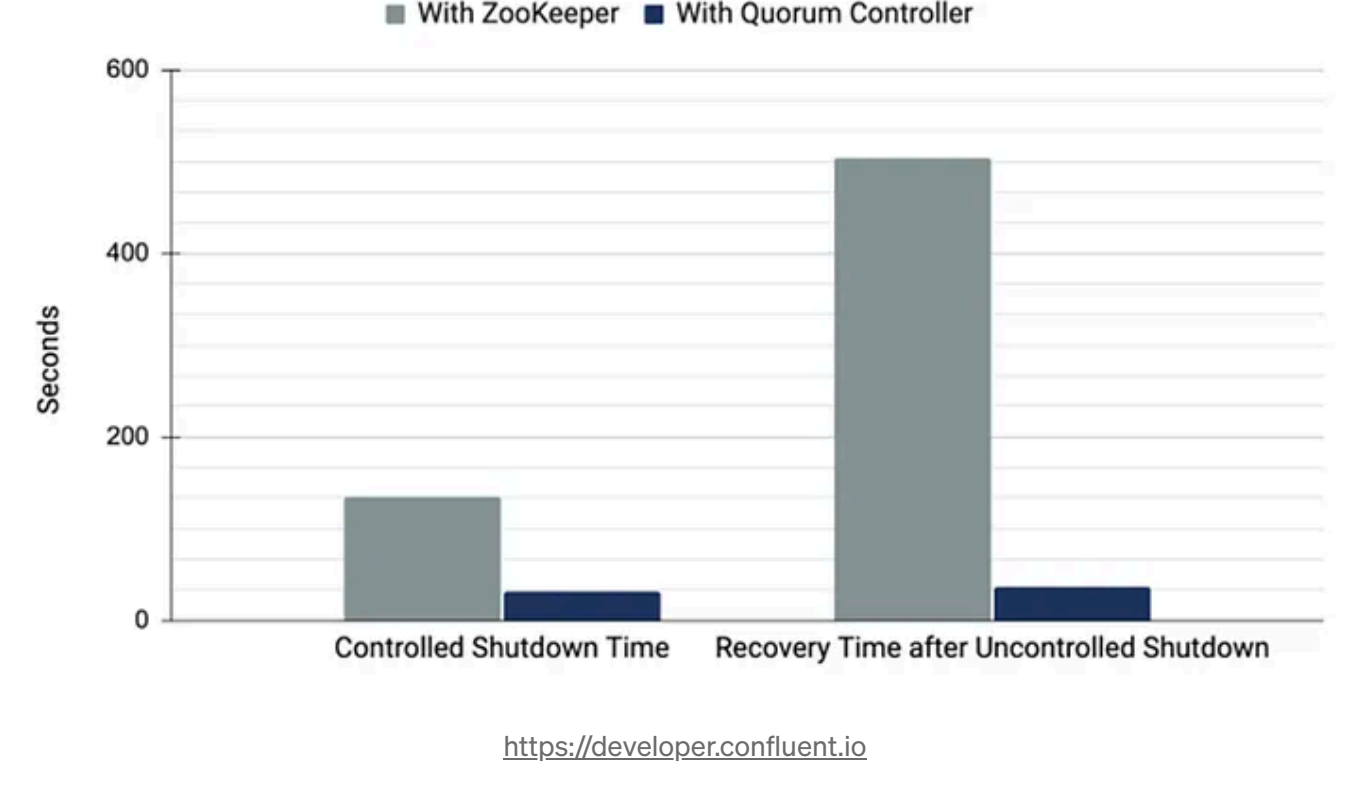
If the leader broker fails, one among the standby controller brokers becomes the active controller. The new leader broker will then synchronize its copy of the metadata with the other brokers.



The below table compares the pros and cons of using ZooKeeper in older Kafka versions versus the benefits of Kafka Raft metadata mode in newer versions:

Feature	ZooKeeper	Kafka Raft metadata mode
Required	Yes	No
Complexity	More complex	Less complex
Performance	Slower	Faster
Scalability	Less scalable	More scalable
Fault tolerance	Highly available	Highly available
Consistency	Consistent view of the cluster to all clients	Consistent view of the cluster to all clients
Coordination	Facilitates coordination among distributed applications	Facilitates coordination among distributed applications
Supported features	Supports all Kafka features	Does not support all Kafka features
Minimum number of brokers required	No minimum number of brokers required	Requires a minimum number of brokers in the cluster
Migration from ZooKeeper	Not recommended	Not recommended

The Kafka cluster metadata is now stored in the Kafka cluster itself, making metadata update operations faster and more scalable. The metadata is also replicated to all the brokers, making failover from failure faster too. The recovery time after controlled and uncontrolled shutdown is also very low compared to the Kafka version with Zookeeper. Kafka has reported significant scalability improvements after switching to Raft. For example, Kafka benchmarks have shown that Raft can scale to clusters with up to 1,000 brokers compared to a few dozen with the Zookeeper.



<https://developer.confluent.io>

Kafka Raft metadata mode, and it requires a minimum number of brokers in the cluster. Migrating existing Kafka deployments from ZooKeeper to Kafka Raft metadata mode is also not recommended.

Though not for metadata management, Apache Kafka still uses ZooKeeper for backward compatibility and other purposes in versions 2.8.0 and beyond.

	ZooKeeper	3.3.x	3.4.x	3.5.x	3.6.x	3.8.x
Kafka	3.5.x					
	3.4.x					
	3.3.x					
	3.2.x					
	3.1.x					
	3.0.x					
	2.8.x					
	2.7.x					
	2.6.x					
	2.5.x					
	2.4.x					
	2.3.x					
	2.2.x					
	2.1.x					
	2.0.x					
	1.1.x					
	1.0.x					
	0.11.0.x					
	0.10.2.x					
	0.9.0.x					
	0.8.x					

Apache Kafka and ZooKeeper Version compatibility.

With the release of Apache Kafka 3.5, **ZooKeeper is now marked deprecated**. Removal of ZooKeeper is planned in the next major release of Apache Kafka (version 4.0), which is scheduled to happen no sooner than April 2024. During the deprecation phase, ZooKeeper is still supported for metadata management of Kafka clusters, but it is not recommended for new deployments. There is a small subset of features that remain to be implemented in KRaft

In the ZooKeeper, the evolution of Kafka's architecture from heavy reliance on the ZooKeeper to the introduction of Kafka Raft metadata mode represents a significant step forward. It simplifies Kafka deployments, enhances performance, and streamlines operations, making Kafka even more versatile for various use cases.

Must READ for Continuous Learning:

- System Design: <https://bit.ly/3S05RGS>
- Head First Design Patterns: <https://amzn.to/3uDN9F6>
- Clean Code: A Handbook of Agile Software Craftsmanship: <https://bit.ly/4Z0W9Zf>
- Java Concurrency in Practice: <https://bit.ly/486v1qg>
- Java Performance: The Definitive Guide: <https://bit.ly/484BAMk>
- Designing Data-Intensive Applications: <https://amzn.to/3NaWUHZ>
- Designing Distributed Systems: <https://amzn.to/487C7NV>
- Clean Architecture: <https://bit.ly/3RwMIWx>
- Kafka — The Definitive Guide: <https://amzn.to/3NHewv8>

Coming An Effective Software Engineering Manager: <https://amzn.to/3NHewv8>